

Recommendation Systems

MKT 566 · Fall 2026

Davide Proserpio

USC Marshall School of Business

Before We Start

Housekeeping

- **Homework 2** (box office) due **Thursday, Oct. 8**
- **Group project**: proposal presentations on **Oct. 13 and 15**
- Next week: **fall recess**, no class (Oct. 6 and 8)
- Today: **recommendation systems**, then a group discussion on Netflix

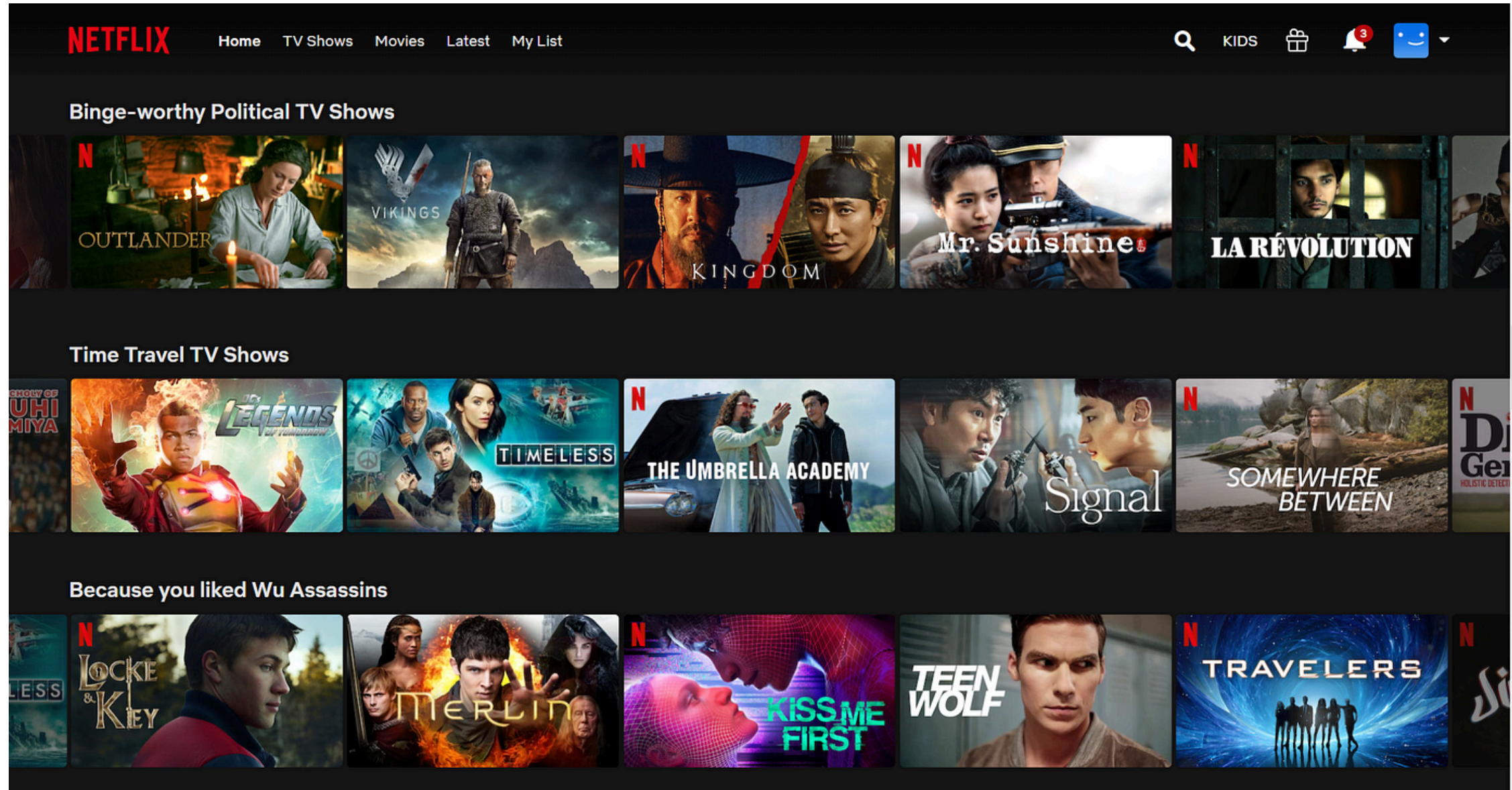
Proposal presentations: how we grade them

Oct. 13 and 15, 15 minutes per group. Send your slides by **Monday, Oct. 12**. The plan does not need to be finished: this is where you get feedback.

What we look at	What a good proposal does
The question	says clearly what you want to find out, and why a marketer would care
The data	says where your data comes from, and shows you can get it
The method	says how you will answer the question, with at least one tool from this class
The talk	clear slides, finishes on time, speaks to the room, handles questions well

Each of the four parts gets **10, 14, 16, 18, or 25 points**, from “needs improvement” to “excellent”, so the total is out of **100**. The proposal counts for **15%** of your group project grade.

Recommendations are everywhere



The Netflix home screen. What is Netflix recommending, and how might it decide?

Recommendations are everywhere

only. Shop items > Terms

No better price found

Bundles with this item

Lifeboost Coffee Whole Bean & Ground Coffe...

-17% \$47.95

Was: \$57.98

[See all bundles](#)

Diet type

✓ USDA Organic

Product details ▼

Additional Details

Small Business

This product is from a small business brand. Support small. [Learn more](#)

[Report an issue with this product or seller](#)

Competitively priced item

Amazon's Choice

Amazon Fresh Organic Fair Trade Sumatra Ground Coffee, Dark Roast, 12 Ounce

12 Ounce (Pack of 1)

★★★★★ (4550)

\$7.72 (\$0.64/ounce) ✓prime

1 sustainability feature

Click to see full view

Ask Rufus

Is this coffee fair trade? Does it have a strong aroma?

An Amazon page for a bag of coffee. On the right, Amazon suggests two things: a **bundle** of the same coffee, and a cheaper coffee from its **own brand**. Why would Amazon show you each one? Who benefits, you or Amazon?

Why recommendations matter for marketing

- Customers **stay longer** when what they see is relevant
- Customers **buy more** when the right product is in front of them
- Customers **come back** when they feel the store “gets” them
- **The risk:** show people only what they already like, and they get stuck in a **filter bubble**. They never discover anything new

Every recommendation is a small marketing decision, made millions of times a day: **which product do we show this person, right now?**

Clustering vs. recommendations

	Clustering (Tuesday)	Recommendations
Goal	put similar customers in groups	guess what one customer will like
Result	a group name (“bargain hunters”)	a list of products, best first, for each customer
Who gets what	everyone in a group gets the same thing	every customer gets their own list
Example	a different email for each group	“because you watched...”

Clustering sorts customers into a few big groups. **Recommendations** tell *one* customer what to try next.

Two ways to build a recommender

	Counting	Learning
Question	which products are bought together?	what will this customer like?
Result	rules like “bread → butter”	a score for every product, for every customer
Example	“frequently bought together”	“because you watched...”
Checked with	support, confidence, lift	how often the list is right; clicks; sales

We start with counting, which needs nothing but a calculator. Then we move to learning, which is what people mean by machine learning.

Counting: What Goes With What

Rules: “people who buy A also buy B”

A **rule** reads $A \rightarrow B$: customers who buy A also tend to buy B. Finding these rules is called **market basket analysis**.

- The data: one row per **basket** (one shopping trip), one column per product: 1 if the product is in the basket, 0 if not
- The method: **count** how often products show up together
- What stores do with the rules:
 - Suggest the second product: “frequently bought together”
 - Put the two products near each other on the shelves
 - Sell them together as a discounted bundle

Five shopping baskets

Basket	Bread	Butter	Milk	Beer	Diapers
1	1	1	0	0	0
2	0	0	1	1	1
3	1	0	1	0	0
4	1	1	1	0	0
5	0	0	0	1	1

1 means the product is in the basket. Which products seem to go together? How would you measure “together” with a number?

Three numbers for a rule

Basket	Bread	Butter
1	1	1
2	0	0
3	1	0
4	1	1
5	0	0

Take the rule **bread** → **butter** and count from the table:

- **Support**: how common is the pair? Baskets with both, out of all baskets. Baskets 1 and 4: **2 of 5 = 40%**
- **Confidence**: when someone buys bread, how often do they also buy butter? Bread is in baskets 1, 3, and 4; two of those have butter: **2 of 3 = 67%**
- **Lift**: how much does buying bread raise the chance of butter? Confidence divided by the share of all baskets with butter (2 of 5 = 40%): **67% ÷ 40% = 1.67**

Lift **1.67**: a bread buyer is 1.67 times as likely to buy butter as a typical shopper. Lift **1**: the two products have nothing to do with each other. **Below 1**: buying one makes the other *less* likely.

Your turn: diapers and beer

A store looks at **100 shopping baskets**:

What is in the basket	Baskets
Diapers and beer	2
Diapers only	1
Beer only	38
Neither	59

Compute the support, confidence, and lift of diapers $\rightarrow$ beer. Would you build a promotion on it?

Diapers and beer: the answer

Number	How to get it	Diapers → beer
Support	baskets with both ÷ all baskets	$2 \div 100 = 2\%$
Confidence	baskets with both ÷ baskets with diapers (2 + 1)	$2 \div 3 = 67\%$
Share with beer	baskets with beer (2 + 38) ÷ all baskets	$40 \div 100 = 40\%$
Lift	confidence ÷ share with beer	$67\% \div 40\% = 1.67$

- Diaper buyers are 1.67 times as likely to buy beer as a typical shopper. That sounds **strong**
- But the support is **2%**: the pattern comes from 2 baskets out of 100. Only 3 shoppers bought diapers at all. Too few to trust, and too few to be worth a promotion

Always read the three numbers together. A big lift on a tiny support is a fun fact, not a strategy. Stores ignore rules below a **minimum support**.

From rules to recommendations

1. **Count** support, confidence, and lift for every pair of products
2. **Drop** the rules that are too rare or too weak
3. When a customer puts A in the basket, **suggest** B, starting with the strongest lift

This is Amazon's "frequently bought together". It needs only past baskets: no model, and nothing about the customer. Its limit: it knows what goes with what, not who *you* are.

Learning: People Like You

Three approaches

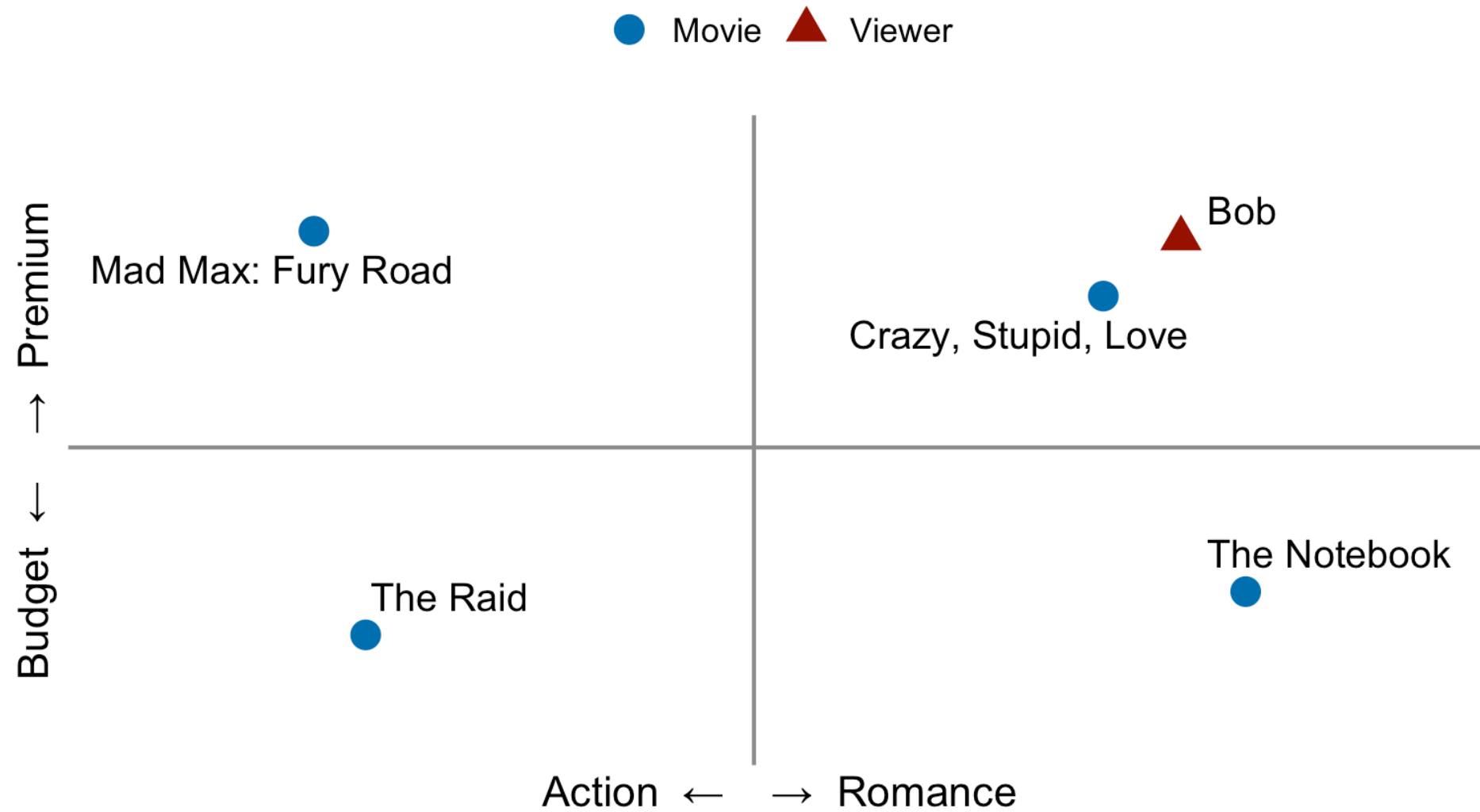
- **Collaborative filtering**: “people like you also liked this”. MBA students who like R tend to like Python too, so suggest Python
- **Content-based filtering**: “more like what you liked”. You liked a sci-fi book, so suggest another sci-fi book
- **Hybrid**: most real platforms mix the two

The core idea: a map of tastes

- Put every customer and every product on a **map**, so that similar ones sit **close together**
- To recommend, find the products **closest** to the customer
- The computer draws the map from what people watch, click, and buy. Its axes capture tastes (genre, price, brand) that nobody had to name
- An **embedding** is the list of numbers that places a customer or a product on the map, one number per axis. On the next map, the movie *Crazy, Stupid, Love* has the embedding (1.35, 1.05): toward romance, and toward a big budget. Similar embeddings mean similar tastes

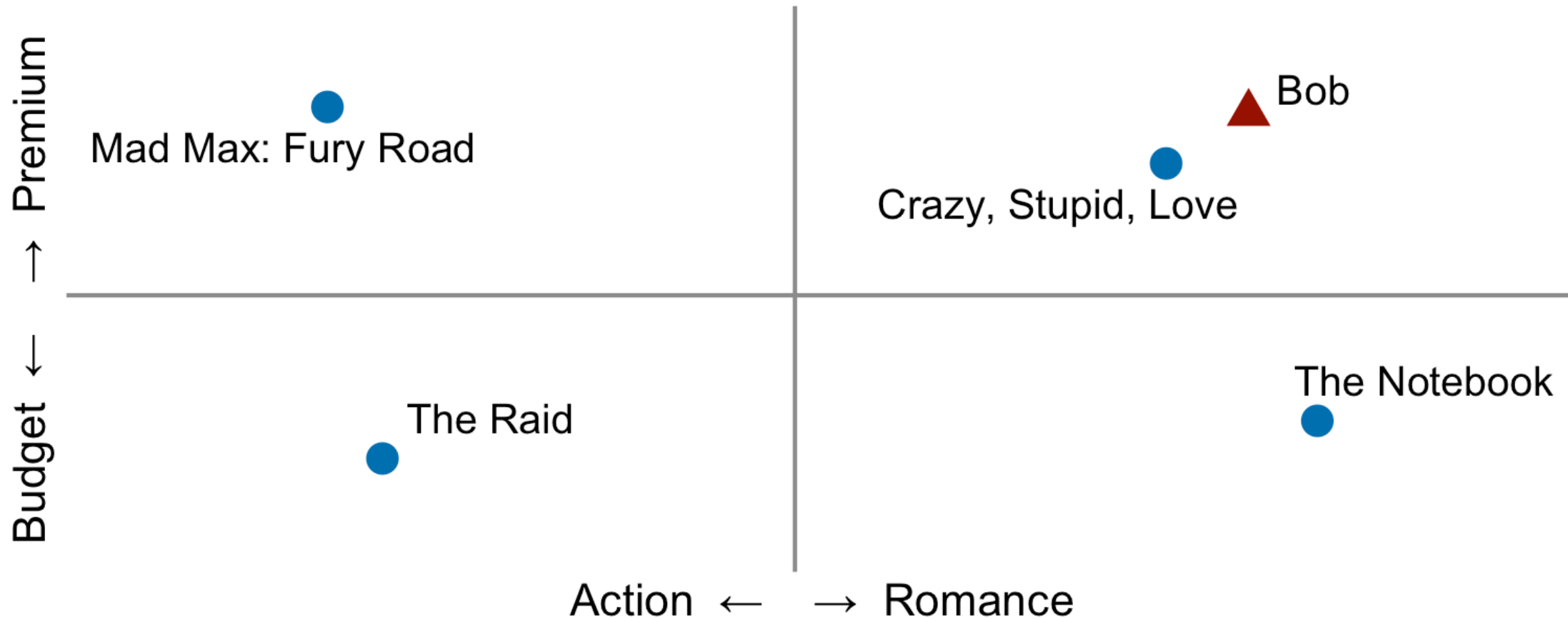
You saw this on Tuesday: **PCA** gave every shopper two coordinates, PC1 and PC2, and we worked out what the axes meant by reading the arrows. Same idea here, learned from millions of clicks.

Movies on a map



A made-up map with four movies and one viewer, Bob. Which movie would you recommend to Bob?

Movies on a map: reading it



- **Crazy, Stupid, Love** is closest to Bob: romance, big budget. Recommend it first, and the action movies last
- A real system does the same with **hundreds of axes** instead of two, and axes nobody has named. The hard part is **drawing the map**

Collaborative filtering

- Guesses what you will like from what **other people** liked
- The bet: people who agreed in the past will agree again
- It only needs to know **who watched, bought, or rated what**. It knows nothing about the movies themselves
- “Collaborative” because everyone’s choices help make everyone else’s recommendations

Five users, five movies

User	The Matrix	Titanic	Toy Story	The Godfather	Inception
1	1	0	1	0	1
2	1	1	0	1	0
3	0	1	1	0	0
4	1	0	0	1	1
5	0	1	0	1	1

1 means the user watched and liked the movie. User 1 has not seen Titanic or The Godfather. Which one would you recommend to User 1, and why?

Five users: the answer

User	Movies in common with User 1	Watched Titanic?	Watched The Godfather?
2	1 (The Matrix)	yes	yes
3	1 (Toy Story)	yes	no
4	2 (The Matrix, Inception)	no	yes
5	1 (Inception)	yes	yes

1. **Compare**: count the movies each user has in common with User 1
2. **Find the closest**: User 4, with 2 movies in common
3. **Recommend** what User 4 liked and User 1 has not seen: **The Godfather**

Real systems compare you with thousands of users and use a finer measure of similarity than a simple count. The logic is the same.

The catch: cold start

Collaborative filtering needs history. No history, no one to compare with. This is called the **cold start** problem:

- A **new customer** has watched nothing: who is this customer similar to?
- A **new movie** has no viewers yet, so it never gets recommended, so it never gets viewers

And most of the table is empty anyway. A real example: **MovieLens**, a movie-recommendation website run by researchers at the University of Minnesota, where users rate movies from 0.5 to 5 stars. In its public sample, **610** users rated **9,724** movies:

- Only **1.7%** of the user-and-movie pairs have a rating
- **66%** of the movies have **5 ratings or fewer**

Source: the MovieLens “small” dataset (GroupLens, University of Minnesota), 100,836 ratings. A new dataset, used only on this slide.

Content-based recommenders

- Recommend products that **look like** what the customer liked before, using what we know about each product: genre, cast, description, price
- Example: you watched a sci-fi thriller, so suggest movies with similar descriptions
- **Plus**: a new movie has a description from day one, so it can be recommended right away
- **Minus**: the **filter bubble**. Everything looks like what you already watched, so you never try anything new

Which one when?

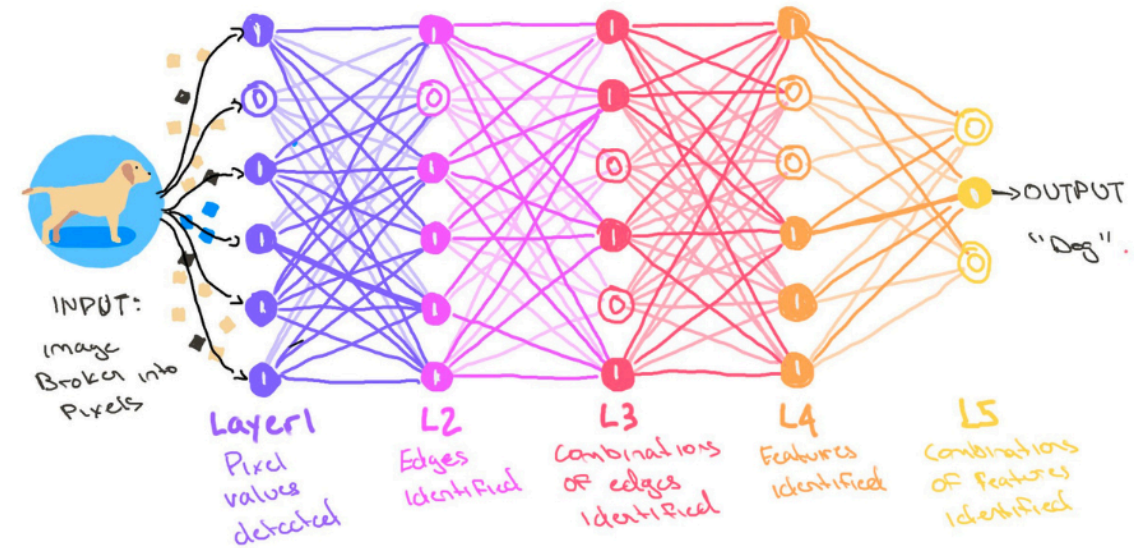
	Collaborative filtering	Content-based
Needs	who watched or bought what	a description of each product
New product	cannot recommend it	can recommend it right away
New customer	cannot recommend	needs a few choices first
Good at	surprises: “people like you liked this”	more of the same
Risk	popular products win	filter bubble

Most platforms use both (**hybrid**): content-based for new products and new customers, collaborative once there is enough history.

The Modern Version: Deep Learning

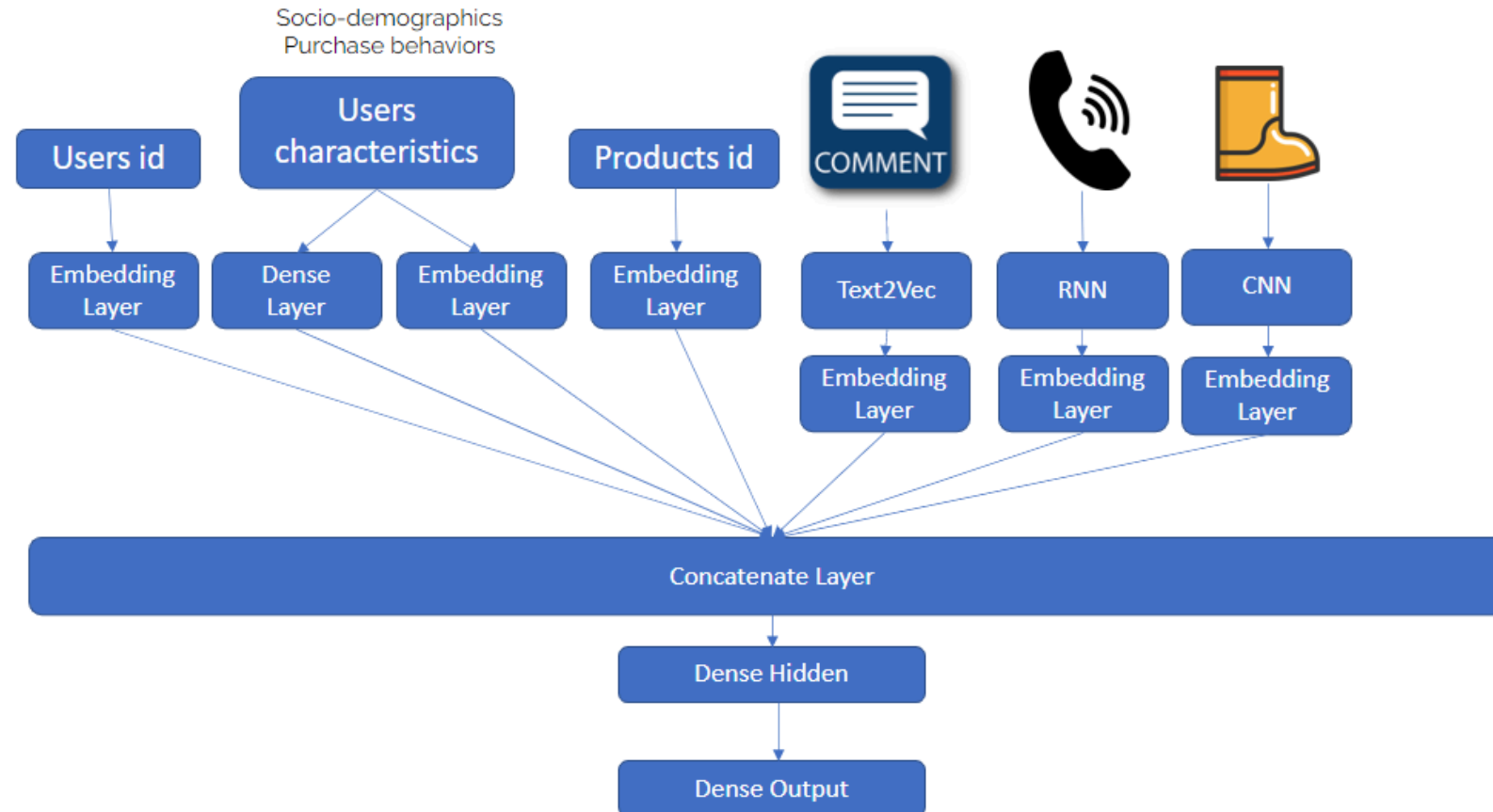
Deep learning, in one picture

- **Deep learning**: computer models built in many **layers**. Each layer spots more complex patterns than the one before
- It works with almost any data: pictures, text, sound, clicks, video
- **ChatGPT, Claude, and Gemini** are deep learning models trained on huge amounts of text: **large language models (LLMs)**



Recognizing a dog: the first layer sees dots of color, the next sees edges, later layers see shapes, and the last one says “dog”.

What deep learning adds



You do not need the details: every kind of data goes in at the top, and one spot on the map comes out at the bottom.

- It draws **much better maps**, because it uses every kind of data: who the customer is, what they bought, their reviews, even product photos
- Content-based, the modern way: an LLM reads each movie's plot and cast and places the movie on the map

Does the Recommendation System Work?

Two ways to test a recommender

	Test 1: on past data	Test 2: with real customers
When	before any customer sees the new recommendations	after the new recommendations go live on the site
How	hide last month, let the system guess, compare with what customers really did	give the new system to half the customers and the old one to the other half
What you measure	precision and recall	clicks, sales, return visits
Why	cheap and fast: catches a bad system before any customer sees it	the real answer: do customers actually click and buy more?

Run test 1 first, because it costs nothing. Run test 2 before you trust the system, because past data cannot tell you how customers will react to new suggestions.

Test 1, on past data: precision and recall

Take one customer's history and **hide last month**. The system looks only at the months before and picks **5 movies** for them. Then we check what the customer **actually liked** last month:

We showed (5)	Matrix	Titanic	Inception	Up	Heat
You liked (4)	Inception	Heat	Coco	Jaws	

The **2 matches** (in red) are the movies we showed **and** you liked:

- **Precision** = matches ÷ movies we showed = $2 \div 5 = 40\%$. Of what we showed, how much did you like?
- **Recall** = matches ÷ movies you liked = $2 \div 4 = 50\%$. Of what you liked, how much did we show?

Two more checks: **variety** (does the list cover much of the catalog?) and **surprise** (does it ever show something less popular that people enjoy?).

Test 2, with real customers: an A/B test

Once the new system is live, split the customers **at random** into two groups:

- **Group A** (half the customers) keeps the **old** recommendations
- **Group B** (the other half) gets the **new** ones

Then compare the two groups on **clicks** (do they click on what we recommend?), **sales** (do they buy, book, or stream?), and **return visits** (do they come back?).

Example: in group A, 5 of every 100 customers click on a recommendation; in group B, 8 of every 100 do. The new system raises clicks from 5% to 8%. Because the split was random, the two groups are alike in everything except the recommender, so the difference comes from the new system. More on experiments later in the course.

Beyond the numbers

- **Fairness:** do small brands and new artists ever get shown, or only the big ones?
- **Explanation:** *should* a customer see **why** something was recommended (“Because you watched...”)? It builds trust, but it also shows how much the company knows about them
- **The long run:** do recommendations build loyalty, or just quick clicks?

Every recommender is built to maximize *something*. Clicks today and loyal customers next year can pull in different directions. Picking the goal is a marketing decision, not a technical one.

In-class assignment: Netflix

In small groups, talk about what Netflix's recommender has to get right. Pick a few questions from each area:

- **The viewer**: something new, or the safe pick? What do you show a brand-new subscriber?
- **Filter bubbles**: how far outside your usual taste should Netflix go?
- **The business**: watch time, keeping subscribers, what shows cost. Should Netflix push its own shows?
- **The data**: what reveals taste best: finishing a show, rewatching, browsing?
- **Ethics and the long run**: privacy, fairness, clicks vs. real satisfaction

Each group shares **one or two things** that surprised you and **one challenge** you see for Netflix. Full questions: [assignment sheet](#).

Wrap-up

- **Counting**: rules find what is bought together. Read support, confidence, and lift **together**
- **Learning**: collaborative filtering (people like you) and content-based filtering (more like what you liked). Most platforms mix both
- Both come down to **similarity**: who or what is close to whom on a map of tastes
- Test a recommender twice: on past data (precision, recall), then with real customers in an A/B test (clicks, sales, return visits). And decide what it should aim for

Required reading: [Netflix's billion-dollar secret](#) and [Marketing automation: recommendation systems](#). Optional: [Two decades of recommender systems at Amazon](#). Next week: fall recess, no class.

Questions?